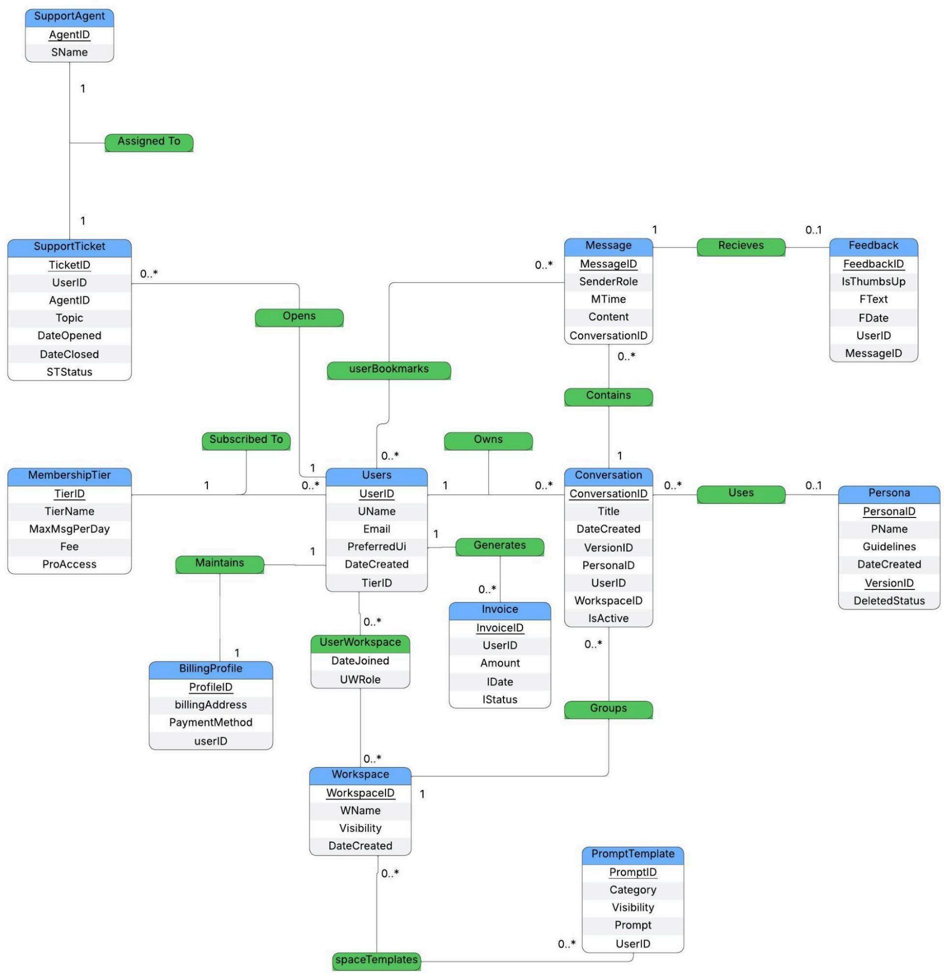
I. Conceptual Database Design

Final E-R Diagram



<u>Design rationale and high–level description of the data model</u>

When designing our database, we wanted to minimize the complexity of relationships while also being able to represent many entities and their characteristics as well as the interactions between them. There are tables for all core multi-attribute entities like messages and conversations, and we focused on utilizing foreign keys to connect entities when their relationships were 1 to many. Initially the idea was to create junction tables between all connected relations, but this was unnecessarily bloated in terms of data manipulation and space. We settled on a composite key with versions for Personas so when one is "deleted" or updated, conversations maintain their integrity. There are many constraints not shown here, ensuring attributes are not null if necessary like for all primary keys. We also decided that membership tiers and agents would be pre-populated relations in the database, since users cannot create those and we must provide options for them to use. Overall, we are pleased with the final design.

ii. Logical database design

MembershipTier:

- TierID (INTEGER, PK)
- TierName (VARCHAR2(15), NOT NULL)
- MaxMsgPerDay (INTEGER, NOT NULL)
- Fee (NUMBER(10,2), NOT NULL)
- ProAccess (INTEGER, DEFAULT 0, CHECK 0 or 1)

Users:

- UserID (INTEGER, PK)
- UName (VARCHAR2(20), NOT NULL)
- Email (VARCHAR2(30), UNIQUE, NOT NULL)
- PreferredUI (VARCHAR2(15))
- DateCreated (DATE, NOT NULL)
- TierID (INTEGER, FK → MembershipTier)

Persona:

- PersonaID (INTEGER, part of PK)
- VersionID (INTEGER, part of PK)
- PName (VARCHAR2(20), NOT NULL)
- Guidelines (VARCHAR2(75))
- DateCreated (DATE, NOT NULL)
- DeletedStatus (INTEGER, DEFAULT 0, 0=active, 1=deleted)

Workspace:

- WorkspaceID (INTEGER, PK)
- WName (VARCHAR2(50), NOT NULL)
- Visibility (VARCHAR2(10), CHECK ‘shared’ or ‘private’)
- DateCreated (DATE, NOT NULL)

Conversation:

- ConversationID (INTEGER, PK)
- Title (VARCHAR2(50))
- DateCreated (DATE, NOT NULL)
- VersionID (INTEGER, FK → Persona)
- PersonaID (INTEGER, FK → Persona)
- UserID (INTEGER, NOT NULL, FK → Users, ON DELETE CASCADE)
- WorkspaceID (INTEGER, FK → Workspace, ON DELETE SET NULL)
- IsActive (INTEGER)

  Foreign key (PersonaID, VersionID) references Persona

Logical database design continued…

Message:
- MessageID (INTEGER, PK)
- SenderRole (VARCHAR2(10), CHECK ‘User’ or ‘AI’)
- MTime (DATE, NOT NULL)
- Content (VARCHAR2(4000), NOT NULL)
- ConversationID (INTEGER, NOT NULL, FK → Conversation, ON DELETE CASCADE)

Feedback:
- FeedbackID (INTEGER, PK)
- IsThumbsUp (INTEGER, NOT NULL, CHECK 0 or 1, 1=thumbs up)
- FText (VARCHAR2(500))
- FDate (DATE, NOT NULL)
- UserID (INTEGER, NOT NULL, FK → Users)
- MessageID (INTEGER, NOT NULL, UNIQUE, FK → Message)

PromptTemplate:
- PromptID (INTEGER, PK)
- Category (VARCHAR2(20))
- Visibility (VARCHAR2(10), CHECK ‘private’ or ‘shared’)
- Prompt (VARCHAR2(250), NOT NULL)
- UserID (INTEGER, NOT NULL, FK → Users)

Invoice:
- InvoiceID (INTEGER, PK)
- UserID (INTEGER, NOT NULL, FK → Users)
- Amount (NUMBER(10,2), NOT NULL)
- IDate (DATE, NOT NULL)
- IStatus (VARCHAR2(10), CHECK ‘paid’ or ‘unpaid’)

SupportAgent:
- AgentID (INTEGER, PK)
- SName (VARCHAR2(20), NOT NULL)

SupportTicket:
- TicketID (INTEGER PK)
- UserID (INTEGER, NOT NULL, FK → Users)
- AgentID (INTEGER, NOT NULL, FK → SupportAgent)
- Topic (VARCHAR2(50), NOT NULL)
- DateOpened (DATE, NOT NULL)
- DateClosed (DATE)
- STStatus (VARCHAR2(15), CHECK ‘Resolved’ or ‘Escalated’)

Logical database design continued…

SpaceTemplates:
- WorkspaceID (INTEGER, FK → Workspace, ON DELETE CASCADE)
- PromptID (INTEGER, FK → PromptTemplate, ON DELETE CASCADE)

Primary key (WorkspaceID, PromptID)

UserWorkspace:
- UserID (INTEGER, FK → Users, ON DELETE CASCADE)
- WorkspaceID (INTEGER, FK → Workspace, ON DELETE CASCADE)
- DateJoined (DATE, NOT NULL)
- UWRole (VARCHAR2(20), CHECK ‘Admin’, ‘Editor’, or ‘Viewer’)

Primary key: (UserID, WorkspaceID)

UserBookmarks:
- UserID (INTEGER, FK → Users, ON DELETE CASCADE)
- MessageID (INTEGER, FK → Message, ON DELETE CASCADE)

Primary key: (UserID, MessageID)

iii. Normalization analysis: All FDs of the table and why the table adheres to 3NF / BCNF

<u>3NF</u>: for every non-trivial FD X -> A that holds in R, either:

1. X is a superkey (set of attributes that includes a CK) of R or
2. A is a prime attribute (attribute that is member of any CK of relation) of R

<u>BCNF</u>: for every non-trivial FD X -> A that holds in R:
X is a superkey (set of attributes that includes a CK) of R or

MembershipTier:
TierID -> TierName, MaxMsgPerDay, Fee, ProAccess
-> TierID is a superkey of R, so BCNF✅

Users:
UserID -> UName, Email, PreferredUI, DateCreated, TierID
Email -> UserID, UName, PreferredUI, DateCreated, TierID
-> UserID is a superkey of R
-> Email is also a superkey of R, so BCNF✅

Persona:
PersonaID, VersionID -> PName, Guidelines, DateCreated, DeletedStatus
-> (PersonaID, VersionID) is a superkey of R, so BCNF✅

Workspace:
WorkspaceID ->WName, Visibility ,DateCreated
-> WorkspaceID is a superkey of R, so BCNF✅

Conversation:
ConversationID -> Title, DateCreated, VersionID, PersonaID, UserID, WorkspaceID, IsActive
-> ConversationID is a superkey of R, so BCNF✅

Message:
MessageID -> SenderRole, MTime, Content, ConversationID
-> MessageID is a superkey of R, so BCNF✅

Feedback:
FeedbackID -> IsThumbsUp, FText, FDate, UserID, MessageID
MessageID -> FeedbackID, IsThumbsUp, FText, FDate, UserID
-> FeedbackID is a superkey of R
-> MessageID is also a superkey of R, so BCNF✅

Normalization analysis continued…

PromptTemplate:
PromptID -> Category, Visibility, Prompt, UserID
-> PromptID is a superkey of R, so BCNF✅

Invoice:
InvoiceID -> UserID, Amount, IDate, IStatus
-> InvoiceID is a superkey of R, so BCNF✅

SupportAgent:
AgentID -> SName
-> AgentID is a superkey of R, so BCNF✅

SupportTicket:
TicketID -> UserID, AgentID, Topic, DateOpened, DateClosed, STStatus
-> TicketID is a superkey of R, so BCNF✅

SpaceTemplates:
No non-trivial functional dependencies, composite PK with no other attributes
-> No non-trivial FDs, so vacuously in BCNF ✅

UserWorkspace:
(UserID, WorkspaceID) -> DateJoined, UWRole
-> (UserID, WorkspaceID) is a superkey of R, so BCNF ✅

UserBookmarks:
No non-trivial functional dependencies, composite PK with no other attributes
-> No non-trivial FDs, so vacuously in BCNF ✅

Since most of our tables had just one primary key and the schema was split extensively to isolate entities to their own relations, they easily fell into BCNF. That one primary key formed a single functional dependency with the PK functionally determining all other attributes and of course was naturally a superkey of the relation to satisfy the definition of BCNF.

iv. Query description

The question our custom query answers is "Given a user ID, list all the workspaces where that user has the 'Admin' role, along with the total number of members (users) in each of those workspaces". This query is implemented in the queryWorkspaceMembers() method. It prompts the user to enter a UserID and then returns a formatted report.

SQL Used:

```
SELECT w.WName AS WorkspaceName, COUNT(uw2.UserID) AS MemberCount
FROM Workspace w
JOIN UserWorkspace uwAdmin ON w.WorkspaceID = uwAdmin.WorkspaceID
LEFT JOIN UserWorkspace uw2 ON w.WorkspaceID = uw2.WorkspaceID
WHERE uwAdmin.UserID = ? AND uwAdmin.UWRole = 'Admin'
GROUP BY w.WName
ORDER BY w.WName
```

This query joins three copies of tables Workspace, UserWorkspace (as uwAdmin), and again UserWorkspace (as uw2). The UserID for the query is provided by the program user, and the query uses COUNT and GROUP BY to produce the results.

This query has utility in a collaborative LLM ecosystem, because workspace admins may need to get a quick overview of their teams and team sizes. So this query has its purpose in workspace management and helps enforce role-based access policies.